

04-Introduction to Programming Languages And Python Fundamentals

Jupyter Environment Setup

- Recommended organizing class materials in uniquely named folders for each batch or session in Jupyter.
- Demonstration of creating/renaming folders and files (emphasis on using underscores, avoiding spaces/hyphens in filenames).
- Rationale for Jupyter: allows holistic, project-based work (data collection, cleaning, modeling) and integrated code/output/visualization.
- Explanation of Jupyter cells, edit and command modes, adding/running/deleting cells, and best practices for naming notebooks.

Fundamentals of Computing (OS & Applications)

- Operating systems (OS) act as the 'brain' of computing devices, managing hardware, software, memory, applications, and user interface.
- OS is essential for user–hardware interaction; without it, applications can't be run or managed efficiently.
- Main memory (RAM) is used for app execution because of speed (much faster than permanent storage); OS is the first program loaded into RAM on startup.
- OS also allocates and manages other resources: input/output devices, network components, etc.

Role and Structure of Programming Languages

- Programming languages allow users to communicate with hardware by writing code or instructions.
- Applications (calculators, notepads, browsers) perform user tasks; their underlying logic is programmed in such languages.
- Each programming language comprises keywords (reserved words) and syntax (rules), both critical for writing correct and functional programs.
- Logic development (the "how" of problem-solving) is as important as knowing syntax/keywords and improves with practice.

Classification & Comparison of Programming Languages

- **Low-level Languages**

: Machine language (binary: 0/1), Assembly language (mnemonics). Usually require architecture-specific knowledge and manual memory management; hard to program and manage, not portable or platform-independent.

- **High-level Languages**

: e.g., C, C++, Java, Python, JavaScript. Designed to be readable and easier for humans, with abstraction from underlying hardware.

- **Why High-Level/Python?**

- Simpler, more readable, and expressive (sample code illustrates shorter syntax than C/Java).
- Code is more manageable (important for large applications with many developers).
- High-level languages (esp. Python) are portable and platform-independent; Java and Python are exemplary here.
- C is fast but less portable/platform-independent than Python/Java; Python prioritizes user-friendliness and manageability over pure speed.
- Python handles memory management, security, and features extensive library/package support.

- No single language is best for all situations; Python chosen for data science due to balance of all above factors.

Intermediate Code and Program Execution

- **Source code:** What the programmer writes.
- **Intermediate code:**
Produced during compilation; e.g., .exe in C (executable), object code in Java, bytecode in Python (.pyc files).
- **Machine code:** Final form (binary) executed by the processor.
- In Python, source code → compiled to bytecode → executed by interpreter.

Translators: Compiler vs Interpreter

- **Compiler:**
 - Checks all code at once; produces intermediate code; will not proceed if errors are found.
 - Used in C and Java; in C, code is compiled directly to executable.
- **Interpreter:**
 - Executes code line by line; gives output immediately; stops execution upon encountering an error (previous lines execute).
 - Python employs both: code is compiled to bytecode, then interpreted.

- **Compile-time vs. Run-time:**

In C/Java, type/data management is handled at compile-time; Python uses interpreter (at run-time, types inferred as code executes).

- Python generally described as interpreted language due to its process and error handling.

Types of Errors in Python

- **Compile-time Errors:**

Checked before program runs (syntax errors, indentation errors); caught by Python's compiler before bytecode is produced.

- **Run-time Errors:**

Discovered as code executes (name errors, value errors, type errors); caught by interpreter.

- **Logical Errors:** Not detected by compiler or interpreter (user/programmer responsibility).

- Importance of accurate syntax/indentation in Python (unlike C/Java, which use braces/semicolons).

Platform Independence and Python Virtual Machine (PVM)

- Python's PVM allows bytecode to be executed on any system with Python/PVM installed, making code portable and platform-independent.
- Platform independence is achieved by generating machine-agnostic bytecode, interpreted by PVM.
- Comparison: C is not platform independent (compiled executables are system-specific), but Python is due to this architecture.

Accessing Class Notes

- Notes and lecture materials are stored in Google Drive; accessible via LMS or shared links as shown at the session's end.
 - Students are encouraged to review notes and familiarize themselves with supplementary materials/links.
-

Notes powered by [CraftNote](#)